Generative AI and Applications

Course Code: 25CS60TP1378-3

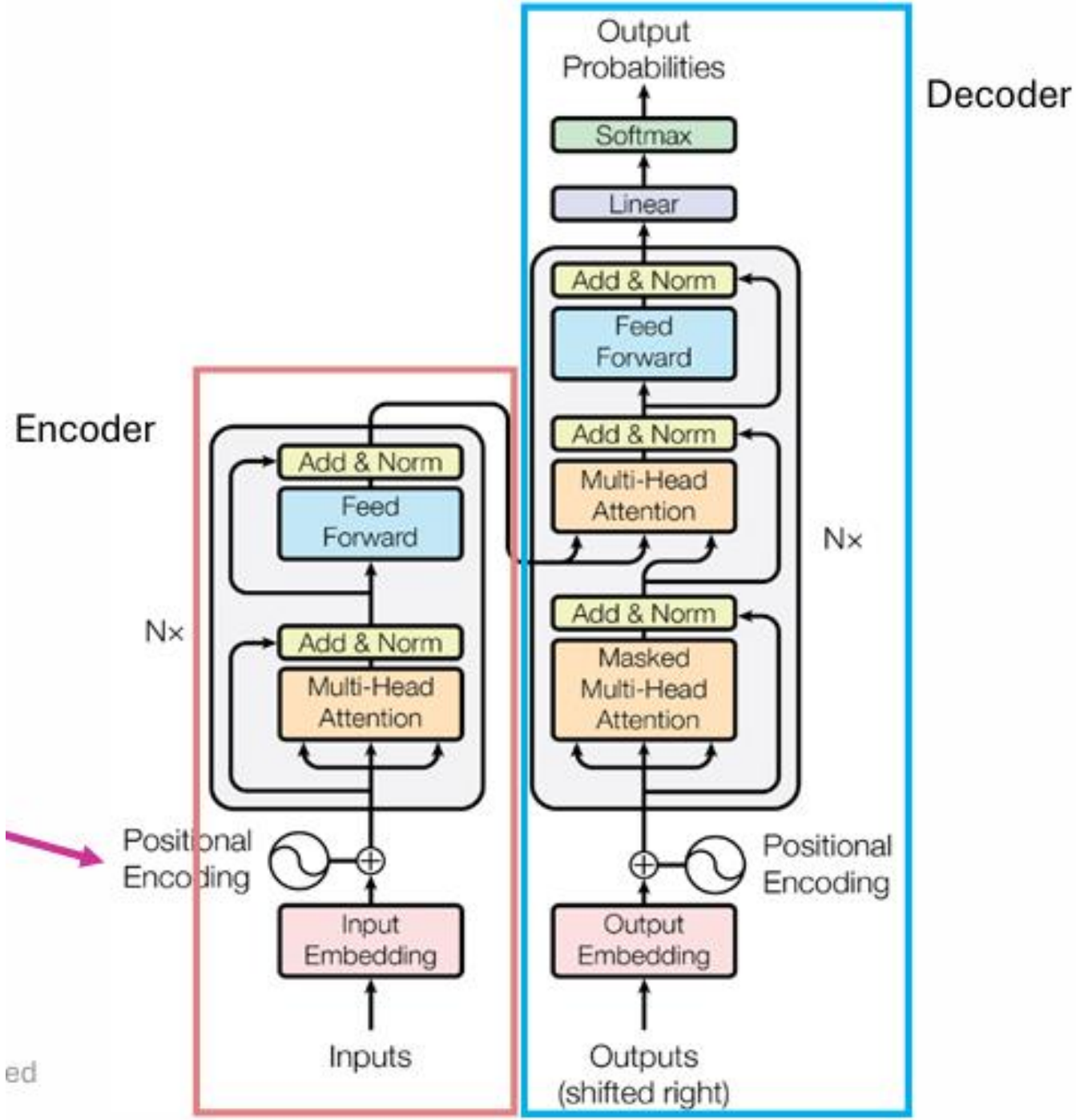
L:3 Hrs, P: 2 Hr, Per Week Total Credits:4

Department of Computer Science and Applications

By Dr. Suhashini Chawrasia

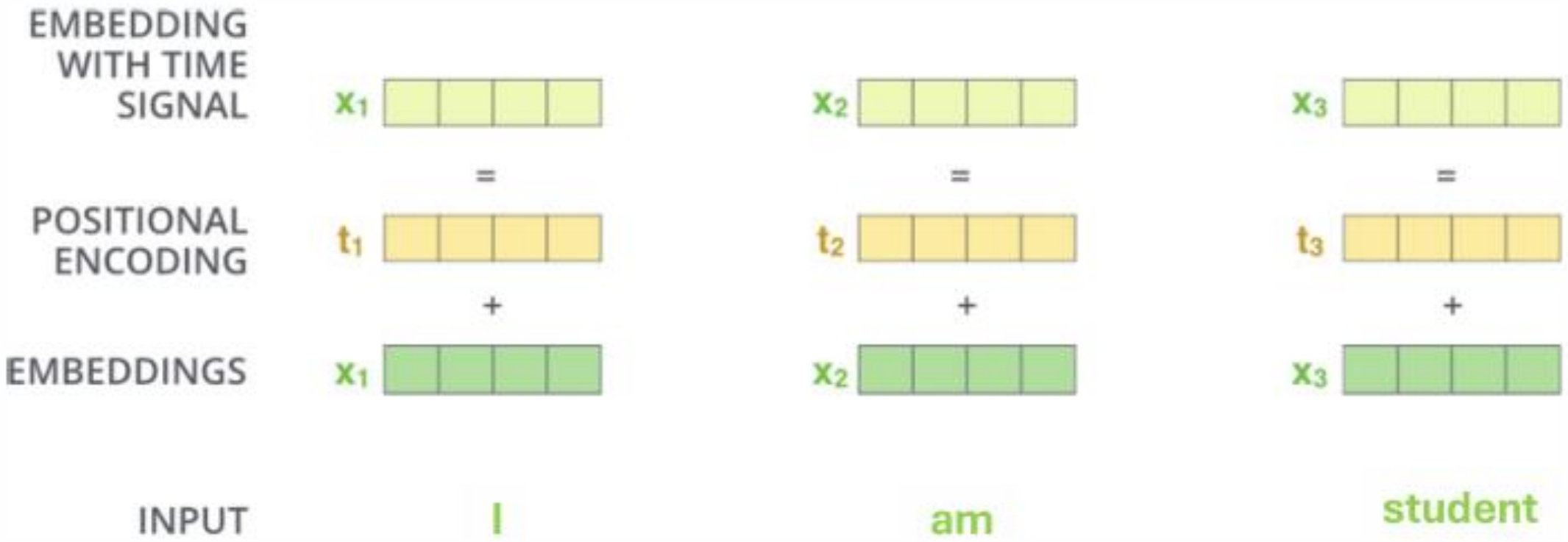
Unit 5

Positional Encoding, Requirements of the Positional Embedding, Formulation of Positional Embedding Vectors, Layer Normalization: Batch Normalization, Internal covariant shift, implementation of Transformers using PyTorch

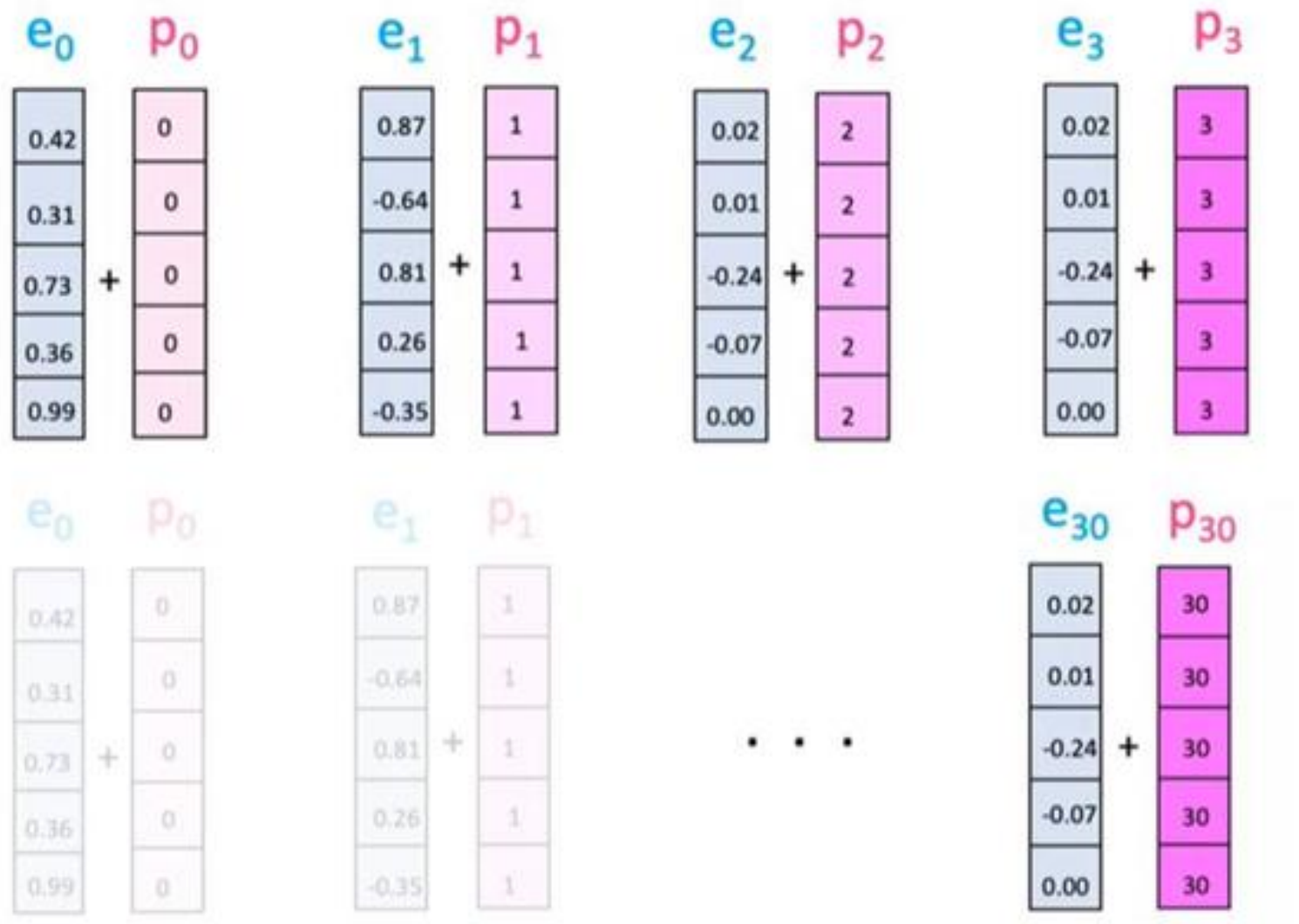

Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Decoder
N×
Positional Encoding
Output Embedding
Outputs (shifted right)
Encoder
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
N×
Positional Encoding
Input Embedding
Inputs

Positional Encoding

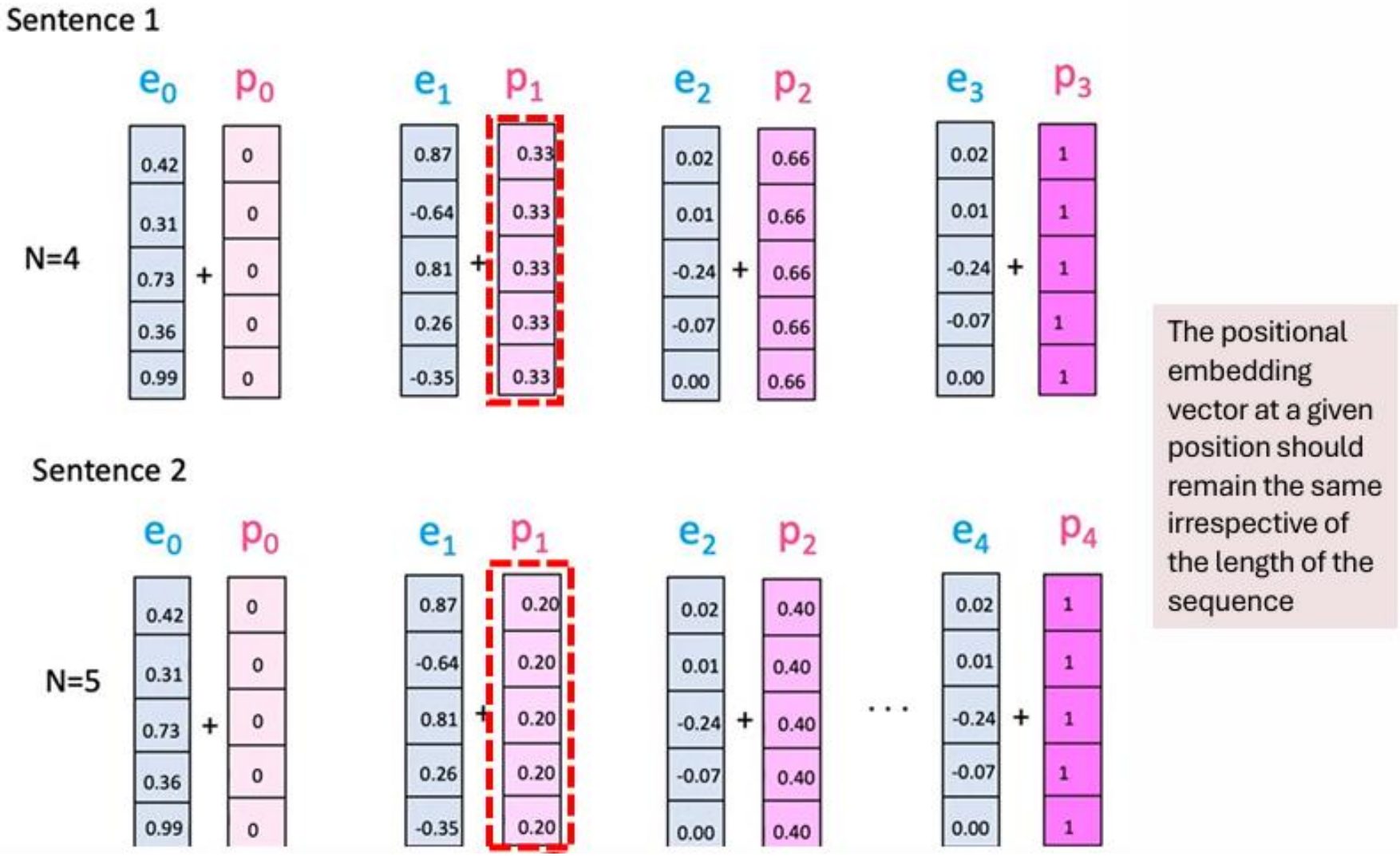
Positional Encoding



Positional Encoding Option 1



Positional Encoding Option 2



The positional embedding vector at a given position should remain the same irrespective of the length of the sequence

Positional Encoding Option 3

0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

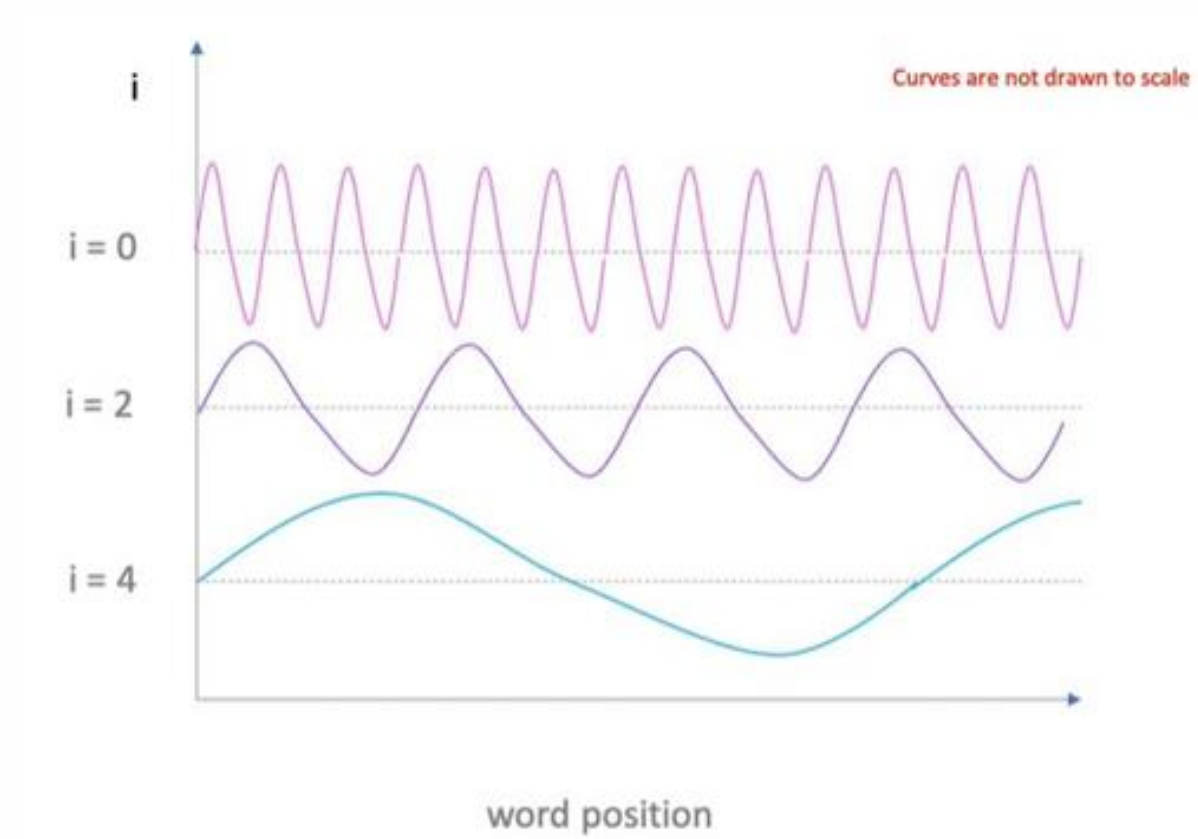
Positional Encoding

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

For $\mathbf{d_{model}}$ **= 512,**
Positional encoding is a 512-dimensional vector
(Note: **Dimension of positional encoding is same as dimension of the word embeddings**)
i = a particular dimension of this vector
pos = position of the word in the sequence

Positional Encoding

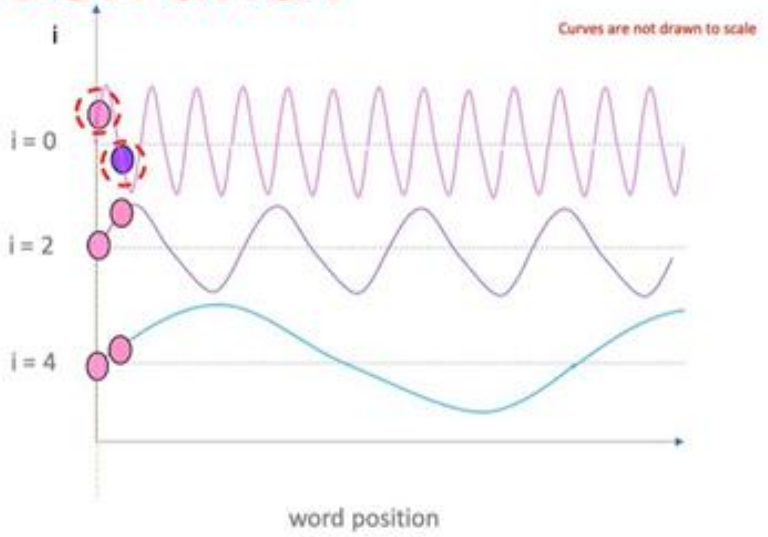


$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

i

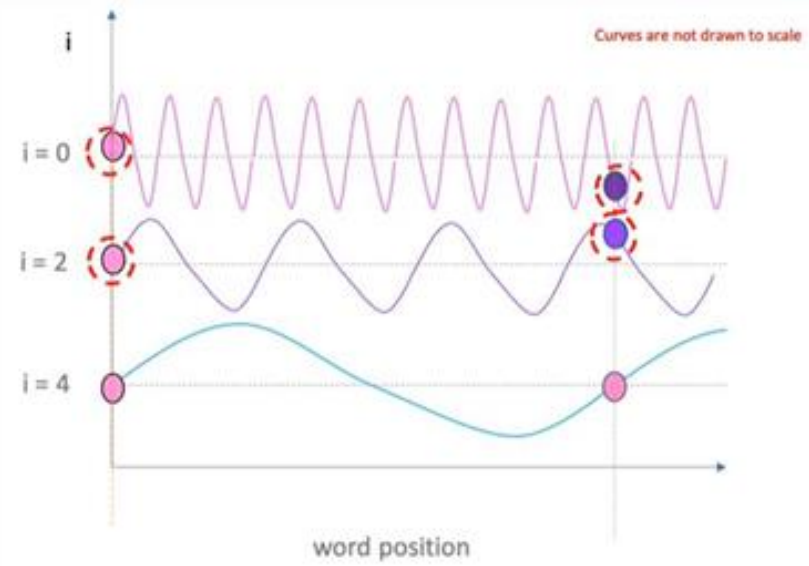
Positional Encoding

Varying Both Position and *i*



$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$ (i)

If two points are close by on the curve, they will remain identical at low frequencies too. It is at the high frequency where their y-axis values differ and we may be able to take them apart.



$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$ (i)

For points which are far apart, we will see them falling apart on the y-axis quite early on

Credits: https://www.youtube.c

Another option
learned positional embeddings

General Properties of positional embeddings

We define the function $\phi(\cdot, \cdot)$ to measure the proximity between positional embeddings.

- **Monotonicity:** The proximity of position embeddings positions decreases when positions are further apart.

$$\forall x, m, n \in \mathbb{N} : m > n \iff \phi(\vec{x}, \overrightarrow{x+m}) < \phi(\vec{x}, \overrightarrow{x+n})$$

- **Translation invariance:** The proximity of embedded positions are translation invariant.

$$\forall x_1, \ldots, x_n, m \in \mathbb{N} : \phi(\vec{x}_1, \overrightarrow{x_1+m}) = \phi(\vec{x}_2, \overrightarrow{x_2+m}) = \cdots = \phi(\vec{x}_n, \overrightarrow{x_n+m})$$

- **Symmetry:** The proximity of embedded positions is symmetric.

$$\forall x, y \in \mathbb{N} : \phi(\vec{x}, \vec{y}) = \phi(\vec{y}, \vec{x})$$

Another option
Rotatory Positional encoding

Absolute Positional Encoding

- Learned from data
- Sinusoidal function (like the original Transformer

Absolute Positional Encoding

$$\mathbb{E}_N = \{\boldsymbol{x}_i\}_{i=1}^N$$

$$\begin{aligned}\boldsymbol{q}_m &= f_q(\boldsymbol{x}_m, m)\\ \boldsymbol{k}_n &= f_k(\boldsymbol{x}_n, n)\\ \boldsymbol{v}_n &= f_v(\boldsymbol{x}_n, n),\end{aligned}$$

$$a_{m,n} = \frac{\exp(\frac{\boldsymbol{q}_m^\intercal \boldsymbol{k}_n}{\sqrt{d}})}{\sum_{j=1}^N \exp(\frac{\boldsymbol{q}_m^\intercal \boldsymbol{k}_j}{\sqrt{d}})}$$

$$\mathbf{o}_m = \sum_{n=1}^N a_{m,n}\boldsymbol{v}_n$$

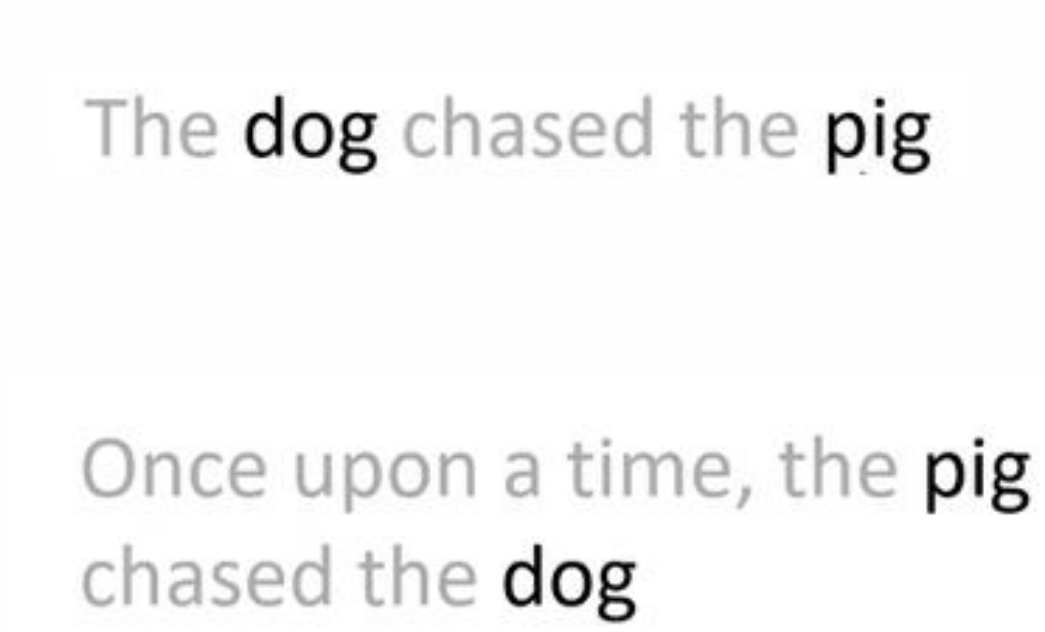
- w_i are tokens and x_i are embeddings

$$f_{t:t\in\{q,k,v\}}(\boldsymbol{x}_i, i) := \boldsymbol{W}_{t:t\in\{q,k,v\}}(\boldsymbol{x}_i + \boldsymbol{p}_i)$$

$$\begin{cases}\boldsymbol{p}_{i,2t} &= \sin(k/10000^{2t/d})\\ \boldsymbol{p}_{i,2t+1} &= \cos(k/10000^{2t/d})\end{cases}$$

- Generate p_i using the sinusoidal function
  - Sinusoidal functions provide continuity between close positions.
  - This also allows for the model to scale to virtually unlimited input sequence length

Relative Positional Encoding



Relative Positional Encoding

$$f_q(\boldsymbol{x}_m) := \boldsymbol{W}_q \boldsymbol{x}_m$$
$$f_k(\boldsymbol{x}_n, n) := \boldsymbol{W}_k(\boldsymbol{x}_n + \tilde{\boldsymbol{p}}_r^k)$$
$$f_v(\boldsymbol{x}_n, n) := \boldsymbol{W}_v(\boldsymbol{x}_n + \tilde{\boldsymbol{p}}_r^v)$$

- $\tilde{p}_r^k, \tilde{p}_r^v \in R^d$ are trainable relative position embeddings.
- r=clip(m−n, r_{min}, r_{max}) is relative distance between positions m and n.
  - Relative position info is not useful beyond a certain distance.

Relative Positional Encoding

Transformer-XL

$$f_{t:t\in\{q,k,v\}}(\boldsymbol{x}_i, i) := \boldsymbol{W}_{t:t\in\{q,k,v\}}(\boldsymbol{x}_i + \boldsymbol{p}_i)$$

- Decompose $q_m^T k_n$

$$\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n + \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{p}_n + \boldsymbol{p}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n + \boldsymbol{p}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{p}_n$$

- Replace absolute position embedding p_n with relative $\tilde{p}_{m-n}$
- Replace absolute position embedding p_m with two trainable vectors u and v independent of query positions.
- W_k is distinguished for content-based and location-based key vectors x_n and p_n, denoted as W_k and $\widetilde{W}_k$

$$\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n + \boldsymbol{x}_m^\intercal \boldsymbol{W}_q^\intercal \widetilde{\boldsymbol{W}}_k \tilde{\boldsymbol{p}}_{m-n} + \mathbf{u}^\intercal \boldsymbol{W}_q^\intercal \boldsymbol{W}_k \boldsymbol{x}_n + \mathbf{v}^\intercal \boldsymbol{W}_q^\intercal \widetilde{\boldsymbol{W}}_k \tilde{\boldsymbol{p}}_{m-n}$$